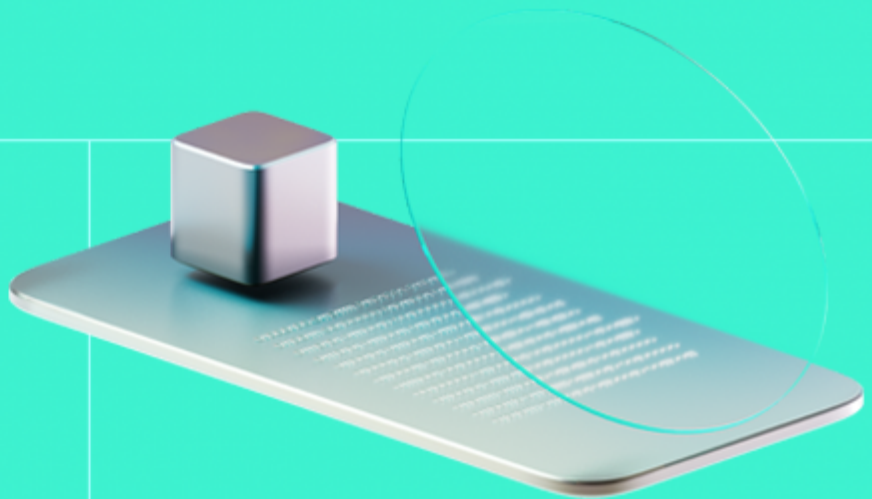




Smart Contract Code Review And Security Analysis Report

Customer: Digital Oro

Date: 21/01/2025



We express our gratitude to the Digital Oro team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Digital Oro is an organization that tokenizes real-world assets into NFTs that can be used as financial digital assets.

Document

Name	Smart Contract Code Review and Security Analysis Report for Digital Oro
Audited By	Kornel Światłowski, Andy Cho
Approved By	Przemyslaw Swiatowiec, Ataberk Yavuzer
Website	https://doi.global/
Changelog	16/12/2024 - Preliminary Report 21/01/2025 - Final Report
Platform	Ethereum
Language	Solidity
Tags	Non-fungible Token (NFT), Token Sales, ERC-721, Lottery
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/digitaloro/sweepstakes
Commit	3a37181023acb05d80f25d24eefe559cddf93b92
Remediation commit	a59038370fc32e0abf88696cd1f495a40723c14e

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

6	5	1	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	2
Low	2

Vulnerability	Severity
F-2024-7595 - Pseudo-Randomness Enables Raffle Outcome Manipulation	High
F-2024-7612 - Lack of Prize Coverage Validation in Raffle Contract	High
F-2024-7645 - Potential Front-Running When DOIToken Is Sold in Secondary Market	Medium
F-2025-8353 - Multiple Payouts Possible Due to Insufficient Validation in FinalizeWave()	Medium
F-2024-7658 - Missing Input Validation for Wave Parameters Causes Denial of Service of Raffle Contract	Low
F-2024-7679 - Absence of Setter Restrictions Allows for Arbitrary Address Updates	Low

Documentation quality

- Functional requirements are provided.
- Technical description is provided.

Code quality

- The development environment is configured.

Test coverage

Code coverage of the project is **67.54%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is partially missed.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	8
Findings	9
Vulnerability Details	9
Observation Details	28
Disclaimers	53
Appendix 1. Definitions	54
Severities	54
Potential Risks	54
Appendix 2. Scope	55
Appendix 3. Additional Valuables	56

System Overview

The `DOI_Token` is a ERC721 token, that allows for creating and managing unique tokens that serve dual purposes: Gold Membership access and raffle participation. It facilitates the minting of tokens in exchange for 100 USDT, with payments securely handled using OpenZeppelin's `SafeERC20` library. The contract includes mechanisms to track token usage for membership claims and raffles, ensuring tokens cannot be reused for these purposes.

The `DOI_Raffle` contract implements a decentralized raffle system where participants use eligible tokens to enter draws conducted in waves. Each wave operates with predefined parameters, including participant limits and prize configurations. The contract ensures fair winner selection through a pseudorandom shuffle of entries, and the winner receives a payout distributed over a specified duration via the `DOI_PayoutManager` contract.

The `DOI_PayoutManager` manages periodic asset distributions, specifically in USDT, to users. It enables authorized parties to initialize, deactivate, reactivate, and claim payouts while enforcing user-specific schedules and limits. The contract integrates with a raffle system for automated payout initialization.

The `DOI_Gold` is an ERC721 token for managing Gold Membership NFTs, designed to reward users of the DOI platform. It integrates with the `DOI_Token` contract to validate ticket eligibility, requiring 20 `DOI_Token` tokens for minting. The contract enforces a maximum supply cap of 5,000 tokens. Upon transfer, the verification status of tokens is reset to ensure validity control.

The `DOI_AdminManager` contract is a utility contract that manages a dynamic list of admin addresses with elevated privileges. It uses an `EnumerableSet` to efficiently store and retrieve admin addresses, ensuring uniqueness and allowing iteration.

The `DOI_AccessControl` contract provides a modular access control mechanism by integrating with an external `DOI_AdminManager` contract. It restricts certain function calls to addresses designated as admins within the `DOI_AdminManager` or the contract owner. The owner, managed via OpenZeppelin's `Ownable2Step`, can update the reference to a new `DOI_AdminManager` contract to maintain flexibility in access control.

The `DOI_RaffleVRF` contract facilitates the secure generation of random numbers using Chainlink VRF v2.5 for a raffle system. It manages requests for random values, tracks their fulfillment status, and stores the results, ensuring that only the designated `DOI_Raffle` contract can initiate these requests.

Privileged roles

The `DOI_Token` contract uses a `Ownable2Step` library from OpenZeppelin to restrict access to important functions. Contract owner can:

- Update the USDT contract address.
- Sets the address for the `DOI_Gold` Membership contract.

- Sets the address for the `DOI_Raffle` contract.
- Withdraw the USDT from the contract to the owner address.

The `DOI_Raffle` contract uses a `DOI_AccessControl` contract to restrict access to important functions.

Contract owner can:

- Updates the `DOI_Token` contract address.
- Updates the `DOI_PayoutManager` contract address.
- Pauses the contract.
- Unpauses the contract.

Admin address can:

- Finalizes the current wave and selects a winner.
- Draw a winner of current wave.
- Starts a new raffle wave.

The `DOI_Gold` contract uses a `DOI_AccessControl` contract to restrict access to important functions. Contract owner can update the `DOI_Token` contract address. Admin address can update the verification status of a token.

The `DOI_PayoutManager` contract uses a `DOI_AccessControl` contract to restrict access to important functions. Contract owner can update the `DOI_Token` contract address. Admin address can update the verification status of a token.

The `DOI_AdminManager` contract uses a `Ownable2Step` library from OpenZeppelin to restrict access to important functions. Contract owner can remove and add a new admin.

The `DOI_AccessControl` contract uses a `Ownable2Step` library from OpenZeppelin to restrict access to important functions. Contract owner can set a new `DOI_AdminManager` contract address.

Potential Risks

- External Calls in Iteration Risks: Making external calls within loops increases the risk of gas exhaustion, potentially leading to failed transactions and reduced contract reliability, especially when processing large datasets.
- Dynamic Array Iteration Gas Limit Risks: The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
- Owner's Unrestricted State Modification: The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases. Like verification status of the **Gold** ERC721 token or addresses of contracts.
- Administrative Key Control Risks: The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.

Vulnerability Details

[F-2024-7595](#) - Pseudo-Randomness Enables Raffle Outcome Manipulation - High

Description:

The `Raffle` contract facilitates a raffle system where users participate by submitting a tokenID from an ERC721-compliant `D0IToken`. Each token is valued at 100 USD. When a token is submitted, it is marked as used, and the user is entered into the raffle. The contract owner invokes the `finalizeWave()` function to determine the winner for a specific wave. Entries for each raffle are stored in the `entries` array.

To determine a winner, the `finalizeWave()` function shuffles the `entries` array using the `shuffleEntries()` function, which relies on the `random()` function for randomization. The winner index is then determined using modulo division.

```
function finalizeWave() external onlyAdmin {
    require(isActive, "Winner has already been drawn");
    require(entries.length >= participantLimit, "Participant limit not reached");

    shuffleEntries(); // Shuffle the entries to randomize their order

    uint256 winnerIndex = random() % entries.length;
    Entry memory winner = entries[winnerIndex];

    {...}
}
```

The `random()` function is implemented as follows:

```
/// @dev Function to generate pseudo-random numbers
function random() internal view returns (uint256) {
    return uint256(keccak256(abi.encodePacked(block.timestamp, block.prevrandao, msg.sender)));
}
```

This implementation is pseudo-random and relies on `block.timestamp` and `block.prevrandao`, both of which are predictable and susceptible to manipulation. Miners can influence these values by delaying the inclusion of the transaction into a block, thereby increasing their

chances of winning. Additionally, the contract owner can simulate potential outcomes before calling the `finalizeWave()` function, enabling the selection of a desired winner. This undermines the fairness of the raffle system.

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]

Status:Fixed

Classification

Impact: 5/5**Likelihood:** 3/5**Exploitability:** Independent**Complexity:** Medium**Severity:** High

Recommendations

Remediation: It is recommended to integrate an external oracle, such as Chainlink VRF, to generate verifiable and tamper-resistant random numbers. This approach would mitigate the predictability and susceptibility to manipulation inherent in the current implementation.

Resolution: The Finding was fixed in commit `fdd11c068b6f01bd9169a4e6cc620b5c8a46cd72`. The pseudo-random mechanism used to determine the raffle winner was replaced by Chainlink VRF (Verifiable Random Function) v2.5.

Evidences

PoC

Reproduce:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Test, console} from "forge-std/Test.sol";
import {DummyUSDT} from "../src/DummyUSDT.sol";
import {DOIToken} from "../src/DOI_Token.sol";
import {AdminManager} from "../src/DOI_AdminManager.sol";
import {PayoutManager} from "../src/DOI_PayoutManager.sol";
```

```

import {Raffle} from "../src/DOI_Raffle.sol";

contract CounterTest is Test {
    DummyUSDT public usdt;
    DOIToken public doiToken;
    AdminManager public adminManager;
    PayoutManager public payoutManager;
    Raffle public raffle;

    address public owner = makeAddr("owner");
    address public addr1 = makeAddr("addr1");
    address public addr2 = makeAddr("addr2");

    function setUp() public {
        vm.startPrank(owner);
        usdt = new DummyUSDT(0);
        doiToken = new DOIToken(address(usdt));
        adminManager = new AdminManager();
        payoutManager = new PayoutManager(address(usdt), address(adminManager
    ));
        raffle = new Raffle(
            address(doiToken), address(payoutManager), address(adminManager),
4, 1e6, 3
        );

        doiToken.setRaffleContractAddress(address(raffle));
        raffle.setPayoutManagerContract(address(payoutManager));
        payoutManager.setRaffleContractAddress(address(raffle));
        payoutManager.setUSDTContractAddress(address(usdt));
        vm.stopPrank();
    }

    function test_POC() public {
        vm.startPrank(owner);
        usdt.mint(addr1, 100);
        usdt.mint(addr2, 300);
        vm.stopPrank();

        vm.startPrank(addr1);
        usdt.approve(address(doiToken), 100e6);
        doiToken.purchaseTokensWithUSDT(1);
        assertEq(doiToken.ownerOf(0), addr1);
        uint256[] memory addr1Tokens = new uint[](1);
        addr1Tokens[0] = 0;
        raffle.enterRaffle(addr1Tokens);
        vm.stopPrank();

        vm.startPrank(addr2);

```

```
usdt.approve(address(doiToken), 300e6);  
doiToken.purchaseTokensWithU
```

[See more](#)

Results:

Logs:

```
Addr1:          0x8C8D736FFa822a05e07F5a46bF29094200Cb7345  
Addr2:          0xE17e8bbfd9A24be5689B0d8a32081fadD8A23B6B  
Winner:         0x8C8D736FFa822a05e07F5a46bF29094200Cb7345  
Predicted winner: 0x8C8D736FFa822a05e07F5a46bF29094200Cb7345
```

Files:

```
[*] Compiling...  
[*] Installing Solc version 0.8.27  
[*] Successfully installed Solc 0.8.27  
[*] Compiling 1 files with 0.8.27  
[*] Solc 0.8.27 finished in 2.01s  
Compiler run successful!  
  
Ran 1 test for test/AuditTestRaffle.sol:CounterTest  
[PASS] test_POC() (gas: 1369870)  
Logs:  
  Addr1:          0x8C8D736FFa822a05e07F5a46bF29094200Cb7345  
  Addr2:          0xE17e8bbfd9A24be5689B0d8a32081fadD8A23B6B  
  Winner:         0x8C8D736FFa822a05e07F5a46bF29094200Cb7345  
  Predicted winner: 0x8C8D736FFa822a05e07F5a46bF29094200Cb7345  
  
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.20ms (866.25µs CPU time)  
Ran 1 test suite in 5.99ms (2.20ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

[F-2024-7612](#) - Lack of Prize Coverage Validation in Raffle Contract - High

Description:

The `Raffle` contract facilitates a raffle system where users participate by submitting a tokenID from an ERC721-compliant `D0IToken`. Each token can be purchased for 100 USD. When a token is submitted, it is marked as used, and the user is entered into the raffle. The winner is determined through the `Raffle::finalizeWave()` function, and the prize is distributed to the winner over a specified period using the `PayoutManager` contract.

In the current implementation, there is no guarantee that the winner will receive the tokens.

There is no validation during wave creation to ensure that the `PayoutManager` contract holds sufficient tokens to cover the prize. This omission can lead to scenarios where the prize cannot be distributed to the winner.

```
function startNewWave(uint256 _participantLimit, uint256 _prizeAmount, uint256 _prizeDuration) external onlyAdmin {
    require(!isActive, "A raffle is already in progress, a new wave can't be started.");
    // Increment to next wave
    currentWave++;

    // Reset entries
    delete entries;

    participantLimit = _participantLimit;
    prizeAmount = _prizeAmount;
    prizeDuration = _prizeDuration;
    isActive = true;

    // Store wave info in history
    waveHistory[currentWave] = WaveInfo({...});
    emit WaveStarted(currentWave, participantLimit, block.timestamp);
}
```

Second, even if sufficient tokens are present at the time of wave creation or user participation, the contract owner can exploit the system. The prize distribution process (`PayoutManager::claimUserPayout()`) can be front-run by the owner, who can withdraw tokens with `PayoutManager::withdrawUSDT()` function.

```
function withdrawUSDT(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(usdtToken.balanceOf(address(this)) >= amount, "Insufficient USDT balance");
    usdtToken.safeTransfer(msg.sender, amount);
}
```

These vulnerabilities can lead to situations where the winner does not receive the tokens they are entitled to, undermining the fairness and integrity of the raffle system.

Assets:

- DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]
- DOI_Raffle.sol [https://github.com/digitaloro/sweepstakes]

Status:

Accepted

Classification

Impact:	5/5
Likelihood:	5/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

It is recommended to enforce validation during wave creation to ensure that the `PayoutManager` contract holds sufficient tokens to cover the prize for the raffle winner. Additionally, prize distribution functions should be secured by restricting unauthorized withdrawals or adding mechanisms to lock prize-related funds until the distribution process is completed.

To fix the issue, it is recommended to introduce a new variable in the `PayoutManager` contract to store the locked tokens allocated for prizes, covering both active waves and ongoing prize distributions over time. This variable should be incremented accordingly by `prizeAmount * prizeDuration` during wave creation and decreased by `totalClaim` when a user claims a prize.

Additionally, the `PayoutManager::withdrawUSDT()` function should be modified to allow withdrawals only for tokens that are not allocated

to any wave, calculated as `usdtToken.balance(address(this)) - allocatedTokens`.

This approach will ensure that the winner receives the entitled prize without interference and that allocated tokens remain securely locked throughout the process.

Resolution:

The Finding was accepted by the Digital Oro team with the following statement:

For this the statement would be: The process for claiming the prize involves a KYC process, which will result in a legal contract binding the user and DOI, outlining all terms. This is why the funds will not be locked for the entire period.

Evidences

PoC

Reproduce:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Test, console} from "forge-std/Test.sol";
import {DummyUSDT} from "../src/DummyUSDT.sol";
import {DOIToken} from "../src/DOI_Token.sol";
import {AdminManager} from "../src/DOI_AdminManager.sol";
import {PayoutManager} from "../src/DOI_PayoutManager.sol";
import {Raffle} from "../src/DOI_Raffle.sol";

contract AuditTest is Test {
    DummyUSDT public usdt;
    DOIToken public doiToken;
    AdminManager public adminManager;
    PayoutManager public payoutManager;
    Raffle public raffle;

    address public owner = makeAddr("owner");
    address public addr1 = makeAddr("addr1");
    address public addr2 = makeAddr("addr2");

    function setUp() public {
        vm.startPrank(owner);
        usdt = new DummyUSDT(0);
        doiToken = new DOIToken(address(usdt));
        adminManager = new AdminManager();
        payoutManager = new PayoutManager(address(usdt), address(adminManager));
    }
}
```

```

));

raffle = new Raffle(
    address(doiToken), address(payoutManager), address(adminManager),
    4, 1e6, 3
);

doiToken.setRaffleContractAddress(address(raffle));
raffle.setPayoutManagerContract(address(payoutManager));
payoutManager.setRaffleContractAddress(address(raffle));
payoutManager.setUSDTContractAddress(address(usdt));
vm.stopPrank();
}

function test_POC() public {
    vm.startPrank(owner);
    usdt.mint(addr1, 100);
    usdt.mint(addr2, 300);
    vm.stopPrank();

    vm.startPrank(addr1);
    usdt.approve(address(doiToken), 100e6);
    doiToken.purchaseTokensWithUSDT(1);
    assertEq(doiToken.ownerOf(0), addr1);
    uint256[] memory addr1Tokens = new uint[](1);
    addr1Tokens[0] = 0;
    raffle.enterRaffle(addr1Tokens);
    vm.stopPrank();

    vm.startPrank(addr2);
    usdt.approve(address(doiToken), 300e6);
    doiToken.purchaseTokensWithUSD

```

[See more](#)

Files:

```

[2629] DummyUSDT::balanceOf(PayoutManager: [0x98B3c60ADE6A87b229Aa7d91ad27c227d54d95C0]) [staticcall]
├─ [Return] 0
├─ [0] VM::assertEq(0, 0) [staticcall]
├─ [Return]
├─ [2599] PayoutManager::allPayouts(0) [staticcall]
├─ [Return] 0, addr2: [0xE17e8bbfd9A24be5689B0d8a32081fadD8A23B6B], 1000000 [1e6], 3, 0, 1, "Payout for wave: 1", true
├─ [0] VM::warp(2592001 [2.592e6])
├─ [Return]
├─ [0] VM::prank(addr2: [0xE17e8bbfd9A24be5689B0d8a32081fadD8A23B6B])
├─ [Return]
├─ [11840] PayoutManager::claimUserPayout(0)
├─ [222] DummyUSDT::decimals() [staticcall]
├─ [Return] 6
├─ [629] DummyUSDT::balanceOf(PayoutManager: [0x98B3c60ADE6A87b229Aa7d91ad27c227d54d95C0]) [staticcall]
├─ [Return] 0
├─ [Revert] revert: Insufficient contract balance
└─ [Revert] revert: Insufficient contract balance

Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 2.29ms (856.03µs CPU time)
Ran 1 test suite in 634.21ms (2.29ms CPU time): 0 tests passed, 1 failed, 0 skipped (1 total tests)

Failing tests:
Encountered 1 failing test in test/AuditTestRaffle.sol:CounterTest
[FAIL, Reason: revert: Insufficient contract balance] test_POC() (gas: 1507900)

```


[F-2024-7645](#) - Potential Front-Running When DOI Token Is Sold in Secondary Market - Medium

Description:

The ERC721-compliant `DOIToken` can be purchased for 100 USDT each and used for participating in raffles or acquiring a Gold Membership NFT. Tokens are marked as used with `DOIToken::useTokenForRaffle()` when entered into a raffle or with `DOIToken::useTokenForMembership()` when used to obtain a Gold Membership by consuming 20 tokens. Once marked as used, tokens cannot be reused for these functionalities.

A potential front-running vulnerability exists when tokens are traded on secondary marketplaces. A malicious seller can list an unused token for sale and monitor the mempool for the corresponding purchase transaction. Before the purchase is finalized, the seller can execute two transactions with higher gas fees to use the token for both raffle entry and Gold Membership acquisition. These front-running transactions are executed before the buyer's transaction, resulting in the buyer receiving a token that is already marked as used.

Assets:

- DOI_Token.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Classification

Impact: 4/5

Likelihood: 2/5

Exploitability: Independent

Complexity: Medium

Severity: Medium

Recommendations

Remediation:

It is recommended to implement a mechanism callable only by the current token owner to temporarily block a token from being used for specific actions such as raffle entry or Gold Membership acquisition. Additionally, transfer functionality should be updated to allow token transfers only when in the blocked state. This approach will mitigate the risk of front-running during token sales on secondary

marketplaces, ensuring tokens retain their intended functionality upon transfer.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. A block token mechanism was introduced to prevent transfers when the token is blocked. The corresponding check was added to the `_update()` function of the ERC20 contract.

[F-2025-8353](#) - Multiple Payouts Possible Due to Insufficient Validation in FinalizeWave() - Medium

Description:

Following the initial audit, a design change was implemented to address issue F-2024-7595, "Pseudo-Randomness Enables Raffle Outcome Manipulation." The pseudo-random mechanism was replaced with Chainlink VRF (Verifiable Random Function) to ensure randomness in selecting the raffle winner. Currently, the contract owner is required to call `DOI_Raffle::drawWinner()` to retrieve a random number from the Chainlink oracle and then call

`DOI_Raffle::finalizeWave()` to select the winner and initiate the payout.

However, due to insufficient validation of the contract state, the `DOI_Raffle::finalizeWave()` function can be invoked multiple times by the contract owner, resulting in multiple payout initializations for the same wave. Currently, the function only checks if the winner has been drawn (`DOI_Raffle::drawWinner()` needs to be called first to pass this check).

```
function drawWinner() external onlyAdmin {
    // Cache variables
    uint256 _currentWave = currentWave;
    uint256 _participantLimit = waveHistory[_currentWave].participantLimit;

    require(!waveHistory[_currentWave].isWinnerDrawn, "Winner has already been drawn");
    require(entries.length >= _participantLimit, "Participant limit not reached");

    // Request random number from VRF contract
    uint256 requestId = doiRaffleVRF.requestRandomWords(false);

    // Set wave inactive and store the request ID
    waveHistory[_currentWave].isWinnerDrawn = true;
    waveHistory[_currentWave].VRFrequestId = requestId;

    emit RandomNumberRequested(requestId);
}

/// @dev Finalizes the current wave and selects a winner.
function finalizeWave() external onlyAdmin {
    // Cache variables
    uint256 _currentWave = currentWave;
    uint256 _prizeAmount = waveHistory[_currentWave].prizeAmount;
    uint256 _prizeDuration = waveHistory[_currentWave].prizeDuration;
    uint256 _VRFrequestId = waveHistory[_currentWave].VRFrequestId;
```

```

        require(waveHistory[_currentWave].isWinnerDrawn, "Winner hasn't been drawn yet");

        // Get generated random number
        (bool fulfilled, uint256[] memory randomWords) = doiRaffleVRF.getRequestStatus(_VRFrequestId);

        require(fulfilled, "Random number was not generated");

        // Select random winner with generated random number
        uint256 winnerIndex = randomWords[0] % entries.length;
        Entry memory winner = entries[winnerIndex];

        // Store wave end info in history
        waveHistory[_currentWave].endTime = block.timestamp;
        waveHistory[_currentWave].winnerAddress = winner.userAddress;
        waveHistory[_currentWave].winnerTokenId = winner.tokenId;
        waveHistory[_currentWave].isActive = false;

        // Set up the winner's payout in the PayoutManager
        string memory info = getMessage("Payout for wave: ", _currentWave);
        payoutManager.initializePayout(winner.userAddress, _prizeAmount, _prizeDuration, info);

        emit WaveEnded(_currentWave, winner.userAddress, winner.tokenId, _prizeAmount, _prizeDuration,
            block.timestamp);
    }

```

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Classification

Impact:	5/5
Likelihood:	3/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

It is recommended to incorporate state validation within the `finalizeWave()` function to ensure it can only be called once per wave. This can be achieved by verifying whether the current wave is active (i.e., `waveHistory[_currentWave].isActive` is true). Once the function is executed, `waveHistory[_currentWave].isActive` is set to false, thereby preventing any further executions for the same wave and ensuring accurate contract state management.

Resolution:

The Finding was fixed in commit `a59038370fc32e0abf88696cd1f495a40723c14e`. A check for the current wave status was added to the `finalizeWave()` function, allowing execution only when the current wave is active and preventing multiple calls to the function.

[F-2024-7658](#) - Missing Input Validation for Wave Parameters Causes Denial of Service of Raffle Contract - Low

Description:

The `Raffle::constructor()` and `Raffle::startNewWave()` functions lack proper input validation for the `_participantLimit`, `_prizeAmount`, and `_prizeDuration` parameters.

The `_prizeAmount` and `_prizeDuration` parameters must have values greater than 0 during new wave creation. In the current implementation, users can use their tickets and participate in a raffle that cannot be ended. If the contract owner attempts to call the `Raffle::finalizeWave()` function to end the active wave, the call reverts during the execution of `PayoutManager::initializePayout()` due to failing checks in the function.

```
function finalizeWave() external onlyAdmin {
    {...}
    payoutManager.initializePayout(winner.userAddress, prizeAmount, prizeDuration, info);
    {...}
}
```

```
function initializePayout(...) external onlyRaffleContract {
    require(monthlyAmount > 0, "Monthly amount must be greater than zero");
    require(durationMonths > 0, "Duration must be greater than zero");
    {...}
}
```

This situation creates a scenario where the active wave cannot be finalized, and a winner cannot be selected. Furthermore, the contract design prevents a new wave from being created while an active wave exists. This results in a complete Denial of Service for the `Raffle` contract.

Additionally, when `_participantLimit` is set to 0, no participants can join the wave, and the wave cannot be ended. The `Raffle::finalizeWave()` function reverts during the `Raffle::shuffleEntries()` call due to an overflow caused by the lack of participants. This also results in a complete Denial of Service for the contract.

```
function shuffleentries() internal {
    uint256 length = entries.length;
    //@audit-info length is 0, no one can enter the current wave
    //@audit-info 0 - 1 will cause overflow
    for (uint256 i = length - 1; i > 0; i--) {
```

```
    {...}
  }
}
```

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]

Status:**Fixed**

Classification

Impact: 4/5**Likelihood:** 3/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** **Low**

Recommendations

Remediation:

It is recommended to implement validation logic for the `_participantLimit`, `_prizeAmount`, and `_prizeDuration` parameters in both the `Raffle::constructor()` and `Raffle::startNewWave()` functions. The `_participantLimit`, `_prizeAmount` and `_prizeDuration` values should be strictly greater than 0. Proper validation will prevent the creation of invalid or non-functional waves and mitigate the risk of Denial of Service in the `Raffle` contract.

Potential fix:

```
constructor(...) DOI_AccessControl(_adminManagerAddress) {
    require(_participantLimit > 0, "_participantLimit must be bigger than 0!");
};

require(_prizeAmount > 0, "_prizeAmount must be bigger than 0!");
require(_prizeDuration > 0, "_prizeDuration must be bigger than 0!");
// current logic
}

function startNewWave(uint256 _participantLimit, uint256 _prizeAmount, uint256
6 _prizeDuration) external onlyAdmin {
    require(_participantLimit > 0, "_participantLimit must be bigger than 0!");
};

require(_prizeAmount > 0, "_prizeAmount must be bigger than 0!");
require(_prizeDuration > 0, "_prizeDuration must be bigger than 0!");
```

```
// current logic  
}
```

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`.
The missing validation of the `_participantLimit`, `_prizeAmount`, and `_prizeDuration` parameters were added to both the `Raffle::constructor()` and `Raffle::startNewWave()` functions.

[F-2024-7679](#) - Absence of Setter Restrictions Allows for Arbitrary Address Updates - Low

Description:

Several contracts within the scope interact with each other and include functions callable exclusively by specific contracts. For instance, the `DOIToken::useTokenForMembership()` function can only be invoked by the `DOIGold` contract. These interactions rely on addresses assigned via dedicated setter functions.

```
constructor(address _usdtTokenAddress, address _adminManagerAddress) DOI_AccessControl(_adminManagerAddress) {
    usdtToken = IERC20(_usdtTokenAddress);
}

/**
 * @dev Updates the raffle contract address.
 * @param _newAddress New raffle contract address.
 */
function setRaffleContractAddress(address _newAddress) external onlyOwner {
    require(_newAddress != address(0), "Invalid address");
    emit RaffleContractUpdated(affleContractAddress, _newAddress);
    raffleContractAddress = _newAddress;
}

/**
 * @dev Updates the USDT contract address.
 * @param _newAddress New USDT contract address.
 */
function setUSDTContractAddress(address _newAddress) external onlyOwner {
    require(_newAddress != address(0), "Invalid address");
    usdtToken = IERC20(_newAddress);
}
```

However, these setter functions are unrestricted and can be called multiple times without time constraints. This creates risks such as breaking contract logic or altering rules accepted by protocol participants. For example, the contract owner could modify the USDT token address (used as a reward token) in the `PayoutManager` contract to a token with significantly lower value, which could harm protocol participants and compromise the protocol's integrity. Additionally, unrestricted setters exacerbate risks if the contract owner's private key is exposed, as an attacker could manipulate key addresses to disrupt operations or misappropriate assets.

Additionally, Address-dependent functions can be executed with uninitialized address variables, as the default value of an `address`

type is `0x0`. This could result in unexpected and undefined contract behavior.

Affected code:

- `DOIToken` : `setUSDTContractAddress()`,
`setGoldMemberContractAddress()`, `setRaffleContractAddress()`,
- `Raffle` : `setDoiTokenContract()`, `setPayoutManagerContract()`,
- `PayoutManager` : `setRaffleContractAddress()`,
`setUSDTContractAddress()`,
- `DOIGold` : `setDoiTokenContract()`

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Gold.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Token.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_PayoutManager.sol
[<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Classification

Impact:	4/5
Likelihood:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

It is recommended to limit the usage of setter functions by allowing each setter to be called only once or by removing the setter functions entirely for addresses that are initialized in the `constructor()`. Addresses set during deployment should be marked as `immutable` to ensure that they cannot be altered post-deployment. These changes will mitigate risks of unauthorized or unintended address modifications. Additional checks should also be introduced to validate that address values are non-zero before executing dependent functions, ensuring consistent and predictable contract behavior.

Potential fix:

```
function setRaffleContractAddress(address _newAddress) external onlyOwner {
    require(_newAddress != address(0), "Invalid address");
    require(affleContractAddress == address(0), "affleContractAddress can be set only once!");
    emit RaffleContractUpdated(affleContractAddress, _newAddress);
    affleContractAddress = _newAddress;
}
```

Resolution:

The Finding was fixed in commit [8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8](#). The setter functions of variables assigned during contract creation were deleted. For the setters of variables not known during contract creation, validations were added to allow setting the given variable only once.

Observation Details

[F-2024-7592](#) - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.0`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- DOI_Gold.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Token.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_PayoutManager.sol
[<https://github.com/digitaloro/sweepstakes>]
- DOI_AdminManager.sol
[<https://github.com/digitaloro/sweepstakes>]
- DOI_AccessControl.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.0`, explicitly define the version with `pragma solidity 0.8.19;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by

referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`.
The pragma directive was locked to version `0.8.27`.

[F-2024-7594](#) - Gas Inefficiency Due to Missing Usage of Solidity Custom Errors - Info

Description:

Starting from Solidity version 0.8.4, the language introduced a feature known as "custom errors". These custom errors provide a way for developers to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, developers often used the `require` statement with string error messages to handle specific conditions or validations. However, every unique string used as a revert reason consumes gas, making transactions more expensive.

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

Assets:

- DOI_AccessControl.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_AdminManager.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_PayoutManager.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Token.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Gold.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to use custom errors instead of reverting strings to reduce increased Gas usage, especially during contract deployment. Custom errors can be defined using the `error` keyword and can include dynamic information.

Current implementation:

```
modifier onlyGoldMemberContract() {  
    require(msg.sender == goldMemberContractAddress, "Not authorized contract");  
    _;  
}
```

Usage of custom errors:

```
// Solidity version 0.8.4 or higher
// custom error declaration
error NotAuthorizedContract();

modifier onlyGoldMemberContract() {
    if(msg.sender != goldMemberContractAddress) revert NotAuthorizedContract(
);
    _;
}
```

or

```
// Solidity version 0.8.26 or higher
// custom error declaration
error NotAuthorizedContract();

modifier onlyGoldMemberContract() {
    require(msg.sender == goldMemberContractAddress, NotAuthorizedContract())
;
    _;
}
```

Resolution:

The Finding was fixed in commit [98e0937948b544befd4082ac019ed96b5dc4bb2a](#).
The revert messages were replaced by custom errors.

[F-2024-7596](#) - Missing Two-step Ownership Transfer Process - Info

Description: The `DOI_AccessControl`, `DOIToken` and `AdminManager` contracts currently utilize the `Ownable` library from OpenZeppelin for managing contract ownership. However, the `Ownable` library lacks a safety mechanism that prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Assets:

- `DOI_Token.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_AdminManager.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_AccessControl.sol` [<https://github.com/digitaloro/sweepstakes>]

Status: Fixed

Recommendations

Remediation: Consider using `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. These contracts prevent the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call. This two-step ownership transfer process adds an additional layer of security to your contract's ownership management.

Resolution: The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. The `Ownable` library was replaced with the `Ownable2Step` library.

F-2024-7610 - Lack Of Cached Values While Reading From Storage Leads To Unnecessary Gas Consumption - Info

Description:

The `mintGoldMembership()` functions in the `DOI_Gold` contract do not cache `totalSupply` value when reading from storage. This results in multiple reads from storage, which is more gas-intensive than reading from memory.

```
function mintGoldMembership() public {  
    ...  
    _safeMint(msg.sender, totalSupply);  
  
    emit GoldMembershipMinted(msg.sender, totalSupply);  
}
```

The same for the `currentWave` variable in the `startNewWave()` function of the `DOI_Raffle` contract.

```
function startNewWave(uint256 _participantLimit, uint256 _prizeAmount, uint256  
6 _prizeDuration) external onlyAdmin {  
    ...  
    waveHistory[currentWave] = WaveInfo({  
        waveNumber: currentWave,  
        ...  
    });  
  
    emit WaveStarted(currentWave, participantLimit, block.timestamp);  
}
```

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Gold.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

Consider caching the value of `totalSupply` and `currentWave` in memory variables when the variables are updated. This will reduce the number of storage reads, leading to more gas-efficient code.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. Temporary memory variables were introduced to prevent redundant

storage reads.

F-2024-7611 - Missing Events in Key Functions - Info

Description:

Sensitive functions that make state changes or update critical variables lack event emissions to log these changes. Event emissions are critical in blockchain applications for tracking state changes, providing transparency, and enabling off-chain systems to monitor contract activity. Without events, changes made by these functions are harder to detect, which could result in diminished auditability and potential misuse going unnoticed.

The following functions lack event emissions:

`DOI_Gold` contract: `setDoiTokenContract()` function.

`DOI_PayoutManager` contract: `setUSDTContractAddress()`, `withdrawUSDT()` functions.

`DOI_Raffle` contract: `setDoiTokenContract()`, `setPayoutManagerContract()` functions.

`DOI_Token` contract: `setGoldMemberContractAddress()`, `setRaffleContractAddress()`, `withdrawUSDT()` functions.

`DOI_Raffle` contract: `constructor()` could have `WaveStarted` event emission like the `startNewWave()` function.

Assets:

- DOI_Gold.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_PayoutManager.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

Add event emissions to all affected functions to log sensitive changes. Emit relevant data for transparency and off-chain monitoring.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. Events were added to the mentioned functions that were not deleted.

[F-2024-7629](#) - Missing Checks for Zero Address - Info

Description:

In Solidity, the Ethereum address

`0x0000000000000000000000000000000000000000000000000000000000000000` is known as the "zero address".

This address has significance because it is the default value for uninitialized address variables and is often used to represent an invalid or non-existent address. The Missing zero address control issue arises when a Solidity smart contract does not properly check or prevent interactions with the zero address, leading to unintended behavior.

The `constructor()` of the `DOIGold`, `PayoutManager`, `Raffle`, and `DOIToken` contracts lack a zero address validation for their input parameters.

Assets:

- `DOI_Raffle.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_Gold.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_PayoutManager.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_Token.sol` [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding `require` statements that ensure address parameters are not the zero address.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. Missing zero address checks were added to the constructors of the `DOIGold`, `PayoutManager`, `Raffle`, and `DOIToken` contracts.

[F-2024-7633](#) - Inconsistent Naming Between Files and Defined Contract Names - Info

Description: Several contracts within the audit scope exhibit inconsistencies between their file names and the names of the contracts defined within those files. For example:

- In the `DOI_Token.sol` file, the contract is named `DOIToken`.
- In the `DOI_Raffle.sol` file, the contract is named `Raffle`.
- In the `DOI_PayoutManager.sol` file, the contract is named `PayoutManager`.
- In the `DOIGold.sol` file, the contract is named `DOIGold`.
- In the `DOI_AdminManager.sol` file, the contract is named `AdminManager`.

These discrepancies between file names and contract names can lead to confusion during code review, development, and integration processes. Misaligned naming conventions increase the risk of misinterpretation and make it more difficult to navigate and reference the codebase accurately.

Assets:

- `DOI_Token.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_PayoutManager.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_Gold.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_Raffle.sol` [<https://github.com/digitaloro/sweepstakes>]
- `DOI_AdminManager.sol` [<https://github.com/digitaloro/sweepstakes>]

Status: Fixed

Recommendations

Remediation: It is recommended to ensure that file names and contract names align consistently. For each file, the name of the file should directly correspond to the contract it defines. Adjustments should be made to either the file names or the contract names to achieve uniformity, depending on the desired naming convention. This practice will improve clarity, reduce potential for errors, and simplify project management across the codebase.

Resolution: The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. Discrepancies between file names and contract names were resolved, aligning file names with contract names.

[F-2024-7634](#) - Missing SafeERC20 Usage in purchaseTokensWithUSDT() Function - Info

Description:

In the `DOIToken` contract, the `purchaseTokensWithUSDT()` function utilizes the `ERC20::transferFrom()` function directly, instead of using the `SafeERC20::safeTransferFrom()` method. While the `SafeERC20` library is imported and employed elsewhere in the contract, the `SafeERC20::safeTransferFrom()` is not used in the `purchaseTokensWithUSDT()`. The absence of `SafeERC20` usage in the `purchaseTokensWithUSDT()` function increases the risk of unintended reverts when interacting with non-compliant `ERC20` tokens, potentially leading to transaction failures and denial of service for users. For example Tether (USDT)'s `transfer()` and `transferFrom()` functions on L1 do not return booleans as the specification requires, and instead have no return value.

```
function purchaseTokensWithUSDT(uint256 tokensToMint) public nonReentrant returns (bool) {
    require(tokensToMint > 0, "You have to buy at least 1 token");

    //Get USDT decimals
    uint256 tokenDecimals = 10**(IERC20Metadata(address(usdtToken)).decimals());
    uint256 totalToPay = tokenPriceInUSDT * tokensToMint * tokenDecimals;

    require(usdtToken.balanceOf(msg.sender) >= totalToPay, "Not enough USDT balance");
    require(usdtToken.allowance(msg.sender, address(this)) >= totalToPay, "ERC20InsufficientAllowance");
    require(usdtToken.transferFrom(msg.sender, address(this), totalToPay), "USDT transfer failed");

    for (uint256 i = 0; i < tokensToMint; i++) {
        uint256 tokenId = tokenCounter;
        _safeMint(msg.sender, tokenId);
        tokenCounter++;
    }
    emit TokensPurchased(msg.sender, tokensToMint, totalToPay);
    return true;
}
```

Assets:

- DOI_Token.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to replace the `ERC20::transferFrom()` call in the `purchaseTokensWithUSDT()` function with the `SafeERC20::safeTransferFrom()` method. This ensures compatibility with non-standard `ERC20` token implementations and reduces the risk of reversion. Adopting `SafeERC20` methods consistently throughout the contract improves reliability and mitigates compatibility issues, particularly when interacting with tokens that do not strictly adhere to the `ERC20` standard.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. The `ERC20::transferFrom()` call in the `purchaseTokensWithUSDT()` function was replaced with `SafeERC20::safeTransferFrom()`.

[F-2024-7644](#) - Violation of Checks-Effects-Interactions Pattern - Info

Description:

The `DOIGold::mintGoldMembership()` and `Raffle::finalizeWave()` functions do not adhere to the Checks-Effects-Interactions (CEI) pattern. The CEI pattern is a best practice in smart contract design, ensuring that contract state updates occur before any external calls. Deviation from this pattern increases the risk of reentrancy attacks, where a malicious contract could exploit external calls to manipulate the flow of execution before state variables are properly updated.

The failure to follow the CEI pattern could compromise the security and integrity of these functions, potentially leading to asset loss or operational failure.

Affected code:

```
function mintGoldMembership() public {
    // Ensure the total supply has not reached the limit
    require(totalSupply < MAX_SUPPLY, "Total supply limit reached");

    // Get the number of tickets owned by the caller
    uint256 ticketCount = doiToken.balanceOf(msg.sender);
    uint256 eligibleTicketCount = 0;

    // Loop through the tickets, check eligibility, and mark them as used in
    // the same loop
    for (uint256 i = 0; i < ticketCount && eligibleTicketCount < 20; i++) {
        uint256 ticketId = doiToken.tokenOfOwnerByIndex(msg.sender, i);
        if (doiToken.isAvailableForMembership(ticketId)) {
            // Mark the ticket as used for membership
            doiToken.useTokenForMembership(ticketId);
            eligibleTicketCount++;
        }
    }

    // Ensure the user has at least 20 eligible tickets
    require(eligibleTicketCount == 20, "Not enough eligible tickets");
    // Mint the Gold Membership token
    totalSupply++;
    _safeMint(msg.sender, totalSupply);
    emit GoldMembershipMinted(msg.sender, totalSupply);
}
```

```
function finalizeWave() external onlyAdmin {
    require(isActive, "Winner has already been drawn");
}
```



```

require(entries.length >= participantLimit, "Participant limit not reached");

shuffleentries(); // Shuffle the entries to randomize their order
uint256 winnerIndex = random() % entries.length;
Entry memory winner = entries[winnerIndex];

// Set up the winner's payout in the PayoutManager
string memory info = getMessage("Payout for wave: ", currentWave);
payoutManager.initializePayout(winner.userAddress, prizeAmount, prizeDuration, info);

// Store wave end info in history

waveHistory[currentWave] = WaveInfo({
    waveNumber : currentWave,
    participantLimit : participantLimit,
    startTime : waveHistory[currentWave].startTime,
    endTime : block.timestamp,
    winnerAddress : winner.userAddress,
    winnerTokenId : winner.tokenId,
    prizeAmount : prizeAmount,
    prizeDuration : prizeDuration,
    isActive : false
});

isActive = false;
emit WaveEnded(currentWave, winner.userAddress, winner.tokenId, prizeAmount, prizeDuration,
                block.timestamp);
}

```

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_Gold.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to restructure the `DOIGold::mintGoldMembership()` and `Raffle::finalizeWave()` functions to comply with the Checks-Effects-Interactions pattern. State variables should be updated before any external calls are made. Additionally, the use of reentrancy guards, such as the `nonReentrant` modifier provided by OpenZeppelin's ReentrancyGuard library, is suggested to provide an extra layer of protection against reentrancy attacks. Adhering to these best

practices will improve the contract's security and mitigate the risk of exploitation.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. Both functions were restructured according to the Checks-Effects-Interactions pattern. Additionally, the `nonReentrant` modifier was added to the `DOIGold::mintGoldMembership()` function.

F-2024-7648 - Natspec Mismatch - Info

Description:

In the `DOI_Gold` contract, the comment within the `mintGoldMembership()` function states that the function permits users with at least 20 eligible tickets to mint a Gold Membership NFT. However, the implemented `require` statement enforces that the `eligibleTicketCount` must be equal to 20. This introduces a contradiction between the comment and the actual implementation, potentially leading to confusion.

```
function mintGoldMembership() public {  
    ...  
    // Ensure the user has at least 20 eligible tickets  
    require(eligibleTicketCount == 20, "Not enough eligible tickets");  
}
```

Furthermore, the `eligibleTicketCount` variable cannot exceed 20 due to a preceding check in the function logic. This restricts the function to scenarios where the ticket count is exactly 20, deviating from the intended behavior as described in the comment.

Assets:

- `DOI_Gold.sol` [<https://github.com/digitaloro/sweepstakes>]

Status:

Accepted

Recommendations

Remediation:

It is recommended to update the function comment to accurately describe the current implementation, which requires the `eligibleTicketCount` to be exactly 20. This adjustment will ensure consistency between the documentation and the actual behavior, preventing potential misunderstandings.

Resolution:

The Finding was accepted by the Digital Oro team.

[F-2024-7649](#) - Redundant Struct Overrides in finalizeWave() Lead to Higher Gas Costs - Info

Description:

The `finalizeWave()` function concludes the current active wave by selecting a random winner, initiating payout distribution, and updating the `waveHistory` mapping, which stores information about each wave in the form of a `WaveInfo` struct.

```
struct WaveInfo {
    uint256 waveNumber;
    uint256 participantLimit;
    uint256 startTime;
    uint256 endTime;
    address winnerAddress;
    uint256 winnerTokenId;
    uint256 prizeAmount;
    uint256 prizeDuration;
    bool isActive;
}
```

The `WaveInfo` struct is overwritten in its entirety during the `finalizeWave()` function execution. Out of the 9 fields in the struct, only 4 are updated (`endTime`, `winnerAddress`, `winnerTokenId`, `isActive`), while the remaining 5 fields are redundantly overridden with the same values they already held.

```
function finalizeWave() external onlyAdmin {
    require(isActive, "Winner has already been drawn");
    require(entries.length >= participantLimit, "Participant limit not reached");

    shuffleentries(); // Shuffle the entries to randomize their order

    uint256 winnerIndex = random() % entries.length;
    Entry memory winner = entries[winnerIndex];

    // Set up the winner's payout in the PayoutManager
    string memory info = getMessage("Payout for wave: ", currentWave);
    payoutManager.initializePayout(winner.userAddress, prizeAmount, prizeDuration, info);

    // Store wave end info in history
    waveHistory[currentWave] = WaveInfo({
        waveNumber: currentWave, //@audit-info same value saved
    });
}
```

```

        participantLimit: participantLimit, //@audit-info same v
        value saved
        startTime: waveHistory[currentWave].startTime, //@audit-info same v
        value saved
        endTime: block.timestamp,
        winnerAddress: winner.userAddress,
        winnerTokenId: winner.tokenId,
        prizeAmount: prizeAmount, //@audit-info same
        value saved
        prizeDuration: prizeDuration, //@audit-info same
        value saved
        isActive: false
    });
    isActive = false;
    emit WaveEnded(currentWave, winner.userAddress, winner.tokenId, prizeAmount, prizeDuration, block.timestamp);
}

```

This redundant update leads to unnecessary gas consumption, increasing the average gas cost of the `finalizeWave()` function. Gas analysis indicates that the current average gas usage of 347,210 can be reduced to 335,574 by optimizing the function to update only the necessary fields in the `WaveInfo` struct. This optimization would save approximately 11,636 gas per transaction.

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to modify the `finalizeWave()` function to selectively update only the required fields in the `WaveInfo` struct. By doing so, redundant storage operations can be avoided, reducing gas costs and improving overall contract efficiency.

Potential fix:

```

waveHistory[currentWave].endTime = block.timestamp;
waveHistory[currentWave].winnerAddress = winner.userAddress;
waveHistory[currentWave].winnerTokenId = winner.tokenId;
waveHistory[currentWave].isActive = false;

```

Resolution:

The Finding was fixed in commit `fdd11c068b6f01bd9169a4e6cc620b5c8a46cd72`. Only the new values are updated instead of the entire `WaveInfo` struct.

[F-2024-7650](#) - Inefficient Gas Usage Due to Redundant Variables in Raffle Contract - Info

Description:

The `Raffle` contract manages the raffle system, where only one active wave can exist at a time. Wave-specific settings, such as `prizeAmount`, `participantLimit`, `prizeDuration`, and the `isActive` flag, are stored in the `WaveInfo` struct.

```
struct WaveInfo {
    uint256 waveNumber;
    uint256 participantLimit;
    uint256 startTime;
    uint256 endTime;
    address winnerAddress;
    uint256 winnerTokenId;
    uint256 prizeAmount;
    uint256 prizeDuration;
    bool isActive;
}
```

Additionally, the `Raffle` contract defines the state variables `participantLimit`, `prizeAmount`, `prizeDuration`, and `isActive` that hold the same values as those stored in the `WaveInfo` struct. These variables are redundant and result in duplicated storage.

```
/// @dev Indicates whether the current wave is active.
bool public isActive;

/// @dev Current wave number.
uint256 public currentWave;

/// @dev Maximum number of participants allowed in the current wave.
uint256 public participantLimit;

/// @dev Monthly prize amount for the current wave.
uint256 public prizeAmount;

/// @dev Prize duration in months for the current wave.
uint256 public prizeDuration;
```

Maintaining the same data in two locations leads to increased gas costs during contract deployment and during the execution of functions that modify these values, as updates must be performed in both the state variables and the struct. Eliminating the redundant

variables and directly accessing the relevant data from the `WaveInfo` struct would improve gas efficiency.

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations**Remediation:**

It is recommended to remove the `participantLimit`, `prizeAmount`, `prizeDuration`, and `isActive` state variables from the `Raffle` contract. Functions should instead retrieve these values directly from the most recent entry in the `WaveInfo` struct. This optimization would reduce storage costs during contract deployment and decrease gas consumption for functions that involve updates to these values.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. The unused state variables `participantLimit`, `prizeAmount`, `prizeDuration`, and `isActive` were removed from the `DOI_Raffle` contract.

[F-2024-7651](#) - Unnecessary Receive Function in Contracts Rejecting Ether Transfers - Info

Description: Receive Ether Function section from Solidity Language Description:

The `receive()` function is executed on a call to the contract with empty `calldata`. This is the function that is executed on plain Ether transfers (e.g. via `.send()` or `.transfer()`).

In the current codebase, the `PayoutManager` contract declare a `receive()` function but include a `revert()` instruction within it. This contract is designed not to accept any native token transfers.

```
/**
 * @dev Fallback function to reject Ether.
 */
receive() external payable {
    revert("This contract does not accept Ether");
}
```

This implementation is redundant because the same behavior can be achieved without declaring the `receive()` function at all. This unnecessary declaration increases the contract's bytecode size and deployment costs.

Assets:

- DOI_PayoutManager.sol
[<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to remove the `receive()` function from `PayoutManager` contract that is not intended to accept native token transfers. Without declaring this function, the contract will automatically reject Ether transfers, achieving the same outcome while optimizing bytecode size and deployment costs.

Resolution:

The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. The `receive()` function containing a revert statement was removed from the `DOI_PayoutManager` contract.

[F-2024-7653](#) - Potential Gas Optimization by Declaring Immutable Variables as Constant - Info

Description: State variables that are not re-assigned after their initial assignment and have a known value before deployment should be declared using the `constant` modifier. This modifier ensures that such variables remain immutable throughout the lifecycle of the contract, thereby improving code readability and clarity.

Furthermore, declaring variables as `constant` can result in gas savings. Accessing a `constant` variable is more efficient because its value is stored in the contract's bytecode and does not involve storage operations, which are comparatively more expensive.

```
/// @notice Price for each token in USDT, set with 18 decimal places.  
uint256 public tokenPriceInUSDT = 100; //100 USD Token price
```

Assets:

- DOI_Token.sol [<https://github.com/digitaloro/sweepstakes>]

Status: Fixed

Recommendations

Remediation: It is recommended to declare the `tokenPriceInUSDT` variable with the `constant` modifier. This change will optimize gas consumption during variable access and clearly convey the immutability of the variable.

Resolution: The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. The `constant` modifier was added to the `tokenPriceInUSDT` variable.

F-2024-7680 - Redundant Imports - Info

Description: The `Ownable` contract is imported into `DOI_Gold` contract but is never used.

Including unused or redundant imports can increase code complexity, reduce readability, and introduce unnecessary dependencies.

Assets:

- `DOI_Gold.sol` [<https://github.com/digitaloro/sweepstakes>]

Status: Fixed

Recommendations

Remediation: Consider removing redundant imports and ensuring that the objects are imported only in the required locations, avoiding unnecessary duplications.

Resolution: The Finding was fixed in commit `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8`. The unused import of the `Ownable` library was removed from the `DOI_Gold` contract.

[F-2024-7681](#) - Misleading Description for Payout::monthlyAmount Field - Info

Description: The `prizeAmount` field in the `Payout`, `WaveInfo`, and `CurrentWaveInfo` structs contains the following description:

```
/// WaveInfo
/// @param prizeAmount Monthly prize amount for the wave.

/// CurrentWaveInfo
/// @param prizeAmount Monthly prize amount for the wave.

/// Payout
/// @param monthlyAmount The monthly payout amount.
```

However, based on the implementation, the values stored in these fields represent the amount without decimal points. This value is passed to the `PayoutManager`, which releases `monthlyAmount * 10^decimals` as the payout. For instance, if the value `1` is stored in `monthlyAmount`, the actual payout will be `1 * 10^6` (assuming `decimals` is set to 6).

```
function claimUserPayout(uint256 userPayoutId) external nonReentrant {
    {...}
    uint256 tokenDecimals = 10**(IERC20Metadata(address(usdtToken)).decimals(
));
    uint256 totalClaim = claimableMonths * payout.monthlyAmount * tokenDecima
ls;
    {...}
}
```

This discrepancy between the description and the implementation may cause confusion during development or maintenance, particularly when interpreting payout amounts.

Assets:

- DOI_Raffle.sol [<https://github.com/digitaloro/sweepstakes>]
- DOI_PayoutManager.sol [<https://github.com/digitaloro/sweepstakes>]

Status:

Fixed

Recommendations

Remediation:

It is recommended to update the description of the `prizeAmount` field in the relevant structs to clearly state that the stored values represent amounts without decimal points, and that they are scaled by 10^{decimals} during payout calculations. This will ensure consistency between the documentation and the implementation, reducing the potential for confusion.

Resolution:

The Finding was fixed in commits `8ffa5a2df927a8a024fc3dc5eb4a752337dcc3e8` and `fdd11c068b6f01bd9169a4e6cc620b5c8a46cd72`. Descriptions were added to all mentioned fields indicating that their values represent the base value without decimals.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/digitaloro/sweepstakes
Commit	3a37181023acb05d80f25d24eefe559cddf93b92
Remediation commit	a59038370fc32e0abf88696cd1f495a40723c14e
Whitepaper	N/A
Requirements	README.md
Technical Requirements	README.md

Asset	Type
DOI_AccessControl.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
DOI_AdminManager.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
DOI_Gold.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
DOI_PayoutManager.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
DOI_Raffle.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract
DOI_Token.sol [https://github.com/digitaloro/sweepstakes]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of Digital Oro, Hacken followed its methodology by performing fuzz-testing on the project's main functions. Foundry, a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, 6 invariants and fuzzing scenarios were tested over 10,000 runs each. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant and Fuzzing	Test Result	Run Count
Mint random amount of DOI Token	Passed	10,000
Mint Gold Token with random amount of DOI Token	Passed	10,000
Mint Gold Token multiple times	Passed	10,000
Claim prize before first month	Passed	10,000
Claim prize between first and second month	Passed	10,000
Claim prize after whole release period	Passed	10,000

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.